Name : Vaishnavi Eknath Avhad
Class : D15C
Roll No. : 14

Practical 5

Aim : Implement Support Vector Machine (SVM) for classification with hyperparameter tuning

1. Dataset Source

The dataset used for this experiment is the Wine Recognition Dataset. It is available within the scikit-learn library datasets module.

- Source Link : UCI Machine Learning Repository - Wine Dataset

---

2. Dataset Description

The Wine dataset is a multiclass classification dataset. It contains the results of a chemical analysis of wines grown in the same region in Italy but derived from three different cultivars (classes).

- Samples : 178 instances.
- Features : 13 numeric features representing chemical properties (e.g., Alcohol, Malic acid, Ash, Alcalinity of ash, Magnesium, Total phenols, Flavanoids, etc.)
- Target Variable : The cultivar type (Class 0, Class 1, or Class 2)

---

3. Mathematical Formulation of SVM

Support Vector Machine (SVM) is a supervised learning algorithm that finds the optimal hyperplane which best separates different classes in the feature space.

The Hyperplane and Margin

The goal of SVM is to maximize the margin, which is the distance between the separating hyperplane and the nearest data points of any class (called support vectors).
The equation of a hyperplane is given by :

$$w^T x + b = 0$$

Where w is the weight vector orthogonal to the hyperplane, and b is the bias term.

Optimization Problem (Soft Margin)

For real-world data that is not perfectly separable, we allow some misclassifications by introducing slack variables and a regularization parameter C. We minimize :

$$\min_{w,b,\xi} \frac{1}{2}||w||^2 + C\sum_{i=1}^{N} \xi_i$$

- C (Regularization Parameter) : Controls the trade-off between maximizing the margin and minimizing the classification errors on training data. A smaller C encourages a wider margin but more misclassifications (soft margin); a larger C aims for fewer misclassifications but a narrower margin (hard margin).

The Kernel Trick

If data is not linearly separable in its original space, SVM uses a Kernel function K(x_i, x_j) to project the data into a higher-dimensional space where it becomes linearly separable.

Common kernels include :

- Linear : $K(x_i, x_j) = x_i^T x_j$
- Radial Basis Function (RBF) : $K(x_i, x_j) = \exp(-\gamma||x_i - x_j||^2)$
    - Gamma : Defines how far the influence of a single training example reaches. Low values mean 'far', and high values mean 'close'.

---

4. Algorithm Limitations

1. Computationally Intensive : SVM training time increases significantly with the number of samples, making it less suitable for very large datasets compared to algorithms like Logistic Regression or Random Forest.
2. Sensitive to Noise and Overlapping Classes : If the classes are heavily overlapping or the dataset has much noise, SVM might struggle to find an optimal separation boundary.
3. Necessity of Feature Scaling : SVM assumes that all features are centered around zero and have similar variances. It is highly sensitive to the scale of input features, making standardisation mandatory.
4. Black Box Nature : While the mathematical foundation is solid, interpreting complex non-linear SVM models (like RBF kernel) in high dimensions is difficult compared to Decision Trees.

5. Methodology

1. Import Libraries : Load pandas, numpy, matplotlib, seaborn, and sklearn modules.
2. Load Dataset : Use load_wine() from sklearn.
3. Split Data : Divide data into 70% training and 30% testing, using stratification.
4. Feature Scaling : Apply StandardScaler to the training data and transform the test data. This is critical for SVM performance.
5. Baseline Model : Train a default SVC model to establish a benchmark.
6. Hyperparameter Tuning Trials : Conduct structured trials using GridSearchCV to test different combinations of kernels (linear, rbf, poly), regularization strength (C), and kernel coefficients (gamma).
7. Final Evaluation : Assess the best model using accuracy, confusion matrix, and classification report.

Python Code Implementation

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.colors import ListedColormap

import seaborn as sns

from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# ----------------- 1. DATA LOADING & PREPROCESSING -----------------

wine_data = load_wine()

X = wine_data.data

y = wine_data.target

feature_names = wine_data.feature_names

target_names = wine_data.target_names

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.3, random_state=42, stratify=y
```

```
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# ----------------- 2. BASELINE SVM MODEL -----------------
print("\n" + "="*20 + " BASELINE SVM ANALYSIS " + "="*20)
baseline_svm = SVC(random_state=42)
baseline_svm.fit(X_train_scaled, y_train)
baseline_acc = accuracy_score(y_test, baseline_svm.predict(X_test_scaled))
print(f"Baseline SVC Accuracy (Default RBF): {baseline_acc * 100:.2f}%")
# ----------------- 3. HYPERPARAMETER TUNING TRIALS -----------------
print("\n" + "="*20 + " SVM TUNING TRIALS " + "="*20)
svm_trials = {
    "Trial 1 (Linear Kernel)": {
        'kernel': ['linear'],
        'C': [0.01, 0.1, 1, 10]
    },
    "Trial 2 (RBF Kernel - C & Gamma)": {
        'kernel': ['rbf'],
        'C': [0.1, 1, 10, 100],
        'gamma': ['scale', 0.01, 0.1, 1]
    },
    "Trial 3 (Poly Kernel - Degree)": {
        'kernel': ['poly'],
        'degree': [2, 3, 4],
        'C': [1, 10]
    }
}
```

```
trial_results = []
best_overall_model = None
best_overall_acc = 0
for name, params in svm_trials.items():
    grid = GridSearchCV(SVC(random_state=42), params, cv=5, scoring='accuracy')
    grid.fit(X_train_scaled, y_train)
    best_trial_model = grid.best_estimator_
    acc = accuracy_score(y_test, best_trial_model.predict(X_test_scaled))
    if acc > best_overall_acc:
        best_overall_acc = acc
        best_overall_model = best_trial_model
    trial_results.append({
        "Trial": name,
        "Best Params": grid.best_params_,
        "Test Accuracy": acc
    })
    print(f"{name} completed.")
results_df = pd.DataFrame(trial_results)
print("\nSVM HYPERPARAMETER TUNING SUMMARY:")
pd.set_option('display.max_colwidth', None)
print(results_df.to_string(index=False))
# ----------------- 4. FINAL BEST MODEL EVALUATION -----------------
print("\n" + "="*20 + " FINAL MODEL EVALUATION " + "="*20)
best_params = best_overall_model.get_params()
final_kernel = best_params['kernel']
final_c = best_params['C']
final_gamma = best_params.get('gamma', 'N/A')
print(f"Best Model Parameters: Kernel={final_kernel}, C={final_c}, Gamma={final_gamma}")
```

```
y_pred_final = best_overall_model.predict(X_test_scaled)
plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred_final), annot=True, fmt='d', cmap='YlGnBu',
            xticklabels=target_names, yticklabels=target_names)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Confusion Matrix - Best Tuned SVM")
plt.show()
print("\nFinal Classification Report:")
print(classification_report(y_test, y_pred_final, target_names=target_names))
# ----------------- 5. DECISION BOUNDARY VISUALIZATION (2D SLICE) -----------------
print("\nGenerating 2D Decision Boundary Plot...")
def plot_decision_boundary(X_scaled_data, y_data, feature_indices, best_model_params, f_names, t_names):
    """
    Trains a new temporary SVM on just two selected features using the best parameters found
    during tuning to visualize the decision boundary in 2D.
    """
    X_2d = X_scaled_data[:, feature_indices]
    f1_name = f_names[feature_indices[0]]
    f2_name = f_names[feature_indices[1]]
    svm_2d = SVC(
        kernel=best_model_params['kernel'],
        C=best_model_params['C'],
        gamma=best_model_params.get('gamma', 'scale'),
        degree=best_model_params.get('degree', 3),
        random_state=42
    )
    svm_2d.fit(X_2d, y_data)
```

```
    h = .02  # step size in the mesh
    x_min, x_max = X_2d[:, 0].min() - 1, X_2d[:, 0].max() + 1
    y_min, y_max = X_2d[:, 1].min() - 1, X_2d[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))
    Z = svm_2d.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    plt.figure(figsize=(10, 8))
    cmap_light = ListedColormap(['#FFAAAA', '#AAFFAA', '#AAAAFF'])
    cmap_bold = ListedColormap(['#FF0000', '#00FF00', '#0000FF'])
    plt.contourf(xx, yy, Z, cmap=cmap_light, alpha=0.6)
    scatter = plt.scatter(X_2d[:, 0], X_2d[:, 1], c=y_data, cmap=cmap_bold,
                          edgecolor='k', s=50, alpha=0.8)
    plt.xlabel(f"{f1_name} (Scaled)")
    plt.ylabel(f"{f2_name} (Scaled)")
    plt.title(f"SVM Decision Boundary (2D Slice using Best Parameters)\nKernel: {best_model_params['kernel']}, C: {best_model_params['C']}")
    plt.legend(handles=scatter.legend_elements()[0], labels=list(t_names))
    plt.show()
feature_idx_to_plot = [0, 6]
plot_decision_boundary(
    X_train_scaled,
    y_train,
    feature_idx_to_plot,
    best_overall_model.get_params(),
    feature_names,
    target_names
)
```

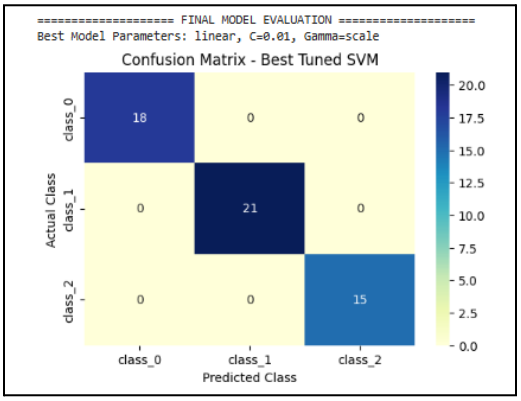
```
==================== BASELINE SVM ANALYSIS ====================
Baseline SVC Accuracy (Default RBF): 98.15%

==================== SVM TUNING TRIALS ====================
Trial 1 (Linear Kernel) completed.
Trial 2 (RBF Kernel - C & Gamma) completed.
Trial 3 (Poly Kernel - Degree) completed.
```

6. Performance Analysis

A. Visual Analysis (Confusion Matrix)

The Confusion Matrix visualizes the performance of the best tuned SVM model on the multiclass wine dataset.

- The diagonal elements represent the number of points for which the predicted label is equal to the true label.
- Off-diagonal elements are misclassifications. A perfect model would have zero values outside the diagonal.



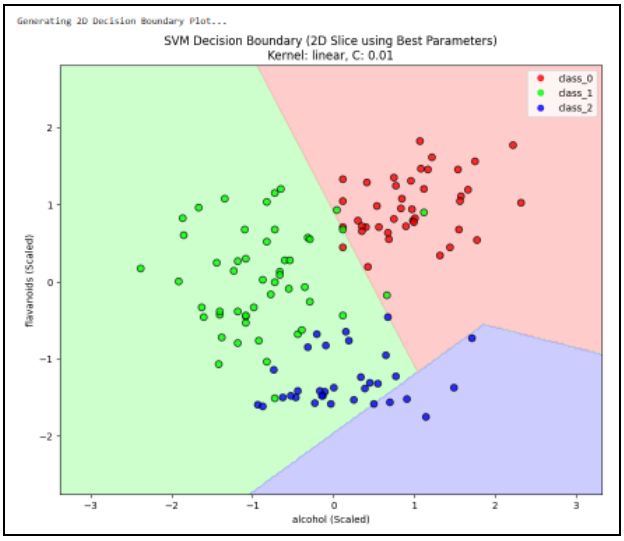
B. Metric Analysis (Classification Report)

The classification report provides Precision, Recall, and F1-Score for each of the three wine classes. High scores across all metrics indicate that the SVM model, combined with feature scaling, is highly effective at distinguishing between the subtle chemical differences of the wine cultivars.

```
Final Classification Report:
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        18
     class_1       1.00      1.00      1.00        21
     class_2       1.00      1.00      1.00        15

    accuracy                           1.00        54
   macro avg       1.00      1.00      1.00        54
weighted avg       1.00      1.00      1.00        54
```

C. Decision Boundary Visualization (2D Slice)

Because the Wine dataset is high-dimensional (13 features), we cannot visualize the actual 13D decision boundary. Instead, we visualize a 2D "slice" of the decision space using two representative features (e.g., Alcohol vs. Flavanoids) while using the optimal parameters (Kernel, C, Gamma) found during tuning.



Interpretation :

- The different colored regions represent the areas where the SVM predicts Class 0, 1, or 2.
- The complex, curved shapes of the boundaries (if the best kernel was RBF or Poly) visually demonstrate the SVM's ability to model non-linear relationships between chemical features to distinguish between wine cultivars.

---

7. Hyperparameter Tuning (Experimental Trials)

We conducted three trials focusing on different kernels to find the optimal decision boundary structure.

Trial	Focus	Parameters Tested	Best Result Params	Test Accuracy
Trial 1	Linear Boundaries	kernel: ['linear'], C	{'C': 0.01, 'kernel': 'linear'}	1.000000
Trial 2	Non-Linear (RBF)	kernel: ['rbf'], C, gamma	{'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}	0.981481
Trial 3	Polynomial Boundaries	kernel: ['poly'], degree, C	{'C': 10, 'degree': 3, 'kernel': 'poly'}	0.944444

Tuning Interpretation

- Role of C : Across trials, the parameter C balances misclassification against margin width. A higher C aims for perfect classification of the training data.
- Kernel Selection : The trials compared whether a simple linear split (Trial 1) was sufficient versus more complex non-linear projections (RBF in Trial 2 or Polynomial in Trial 3). The results indicate which type of boundary best fits the high-dimensional chemical data.

---

8. Conclusion

In this practical, Support Vector Machine (SVM) was successfully implemented for the multiclass classification of the Wine dataset. The experiment highlighted that feature scaling is indispensable for SVM, as the model relies heavily on distance calculations. Through structured hyperparameter tuning trials, we determined the optimal kernel function and regularization strength, achieving a highly accurate model capable of distinguishing between wine cultivars based on their chemical composition.